Warehouse Testing Report

CODE//CONNECT'26 - NexusMart Data Analytics Module

Prepared by: Awan (Person C, Warehouse lead) | Date: 18 October 2026

1. Purpose and scope

This document records how the PostgreSQL warehouse (star schema in the **mart** schema) was built and tested on the real cleaned NexusMart data, which tools were used, and the exact commands that were run. It covers the database and warehouse layer only.

Summary of outcomes

Area	Result	Status
Warehouse SQL run on real data (01 to 05)	All SQL files and loader ran with run_all.ps1	Verified
Quality checks (04_checks.sql)	10 of 10 checks passed (failed_rows = 0)	Verified
KPI view (mart.v_kpi_overall)	Matches the earlier v1 build exactly	Verified
Business-question views Q1-Q5	Created and sanity-checked (category net revenue sums to the KPI Net Revenue)	Verified
Git: commit, push, Pull Request #4, merge to main	Done (merge commit 2e012a4)	Verified

2. Tools used

Tool	What it was used for
Windows PowerShell	Running all commands (scripts, Docker, Git)
Docker Desktop and Docker Compose	Running PostgreSQL in a container (local file docker-compose.wh-local.yaml, port 5433, container cc26-wh-1)
PostgreSQL and psql	Warehouse database (db wh, user wh); SQL scripts and queries executed with psql inside the container
Python 3.14, pandas 3.0.6, psycopg2-binary	Cleaning (teammate B) and sql/load_staging.py, which loads out/ CSVs into Postgres
SQL (PostgreSQL dialect)	Schemas, dimensions, fact table, checks, KPI and question views
Git and GitHub	Branch warehouse, commits, push, Pull Request, merge into main

3. Environment and conventions

Local repo folder: `C:\Users\STAR HIVE\cc26`. Database credentials are kept only in `.env` (variable WH_PW) and are never committed or printed. Only the folders sql/ and docs/ are committed from the warehouse branch.

Every new PowerShell session first sets the compose file, otherwise run_all.ps1 and docker compose look for the wrong file:

```
$env:COMPOSE_FILE="docker-compose.wh-local.yaml"
```

4. Step-by-step testing procedure

Step 1 - Start the local PostgreSQL container

The warehouse database runs from a separate, untracked compose file. Confirm the container and port:

```
$env:COMPOSE_FILE="docker-compose.wh-local.yaml"
docker ps --format "table {{.Names}}\t{{.Ports}}"
```

Expected: container `cc26-wh-1` publishing port 5433.

Step 2 - Run the whole warehouse build on the real data

From the repo root, one script runs the loader and all SQL files in order (staging load, 01_schemas, 02_dims, 03_fact, 04_checks, 05_metrics):

```
powershell -ExecutionPolicy Bypass -File sql\run_all.ps1
```

Files executed by the script:

File	Role
sql/load_staging.py	Loads the cleaned CSVs from out/ into schemas clean and audit (TEXT columns, replace on every run)
sql/01_schemas.sql	Creates schemas raw, clean, audit, mart
sql/02_dims.sql	Builds dim_date, dim_customer, dim_product, dim_seller, dim_location (each with an Unknown member, key -1)
sql/03_fact.sql	Builds mart.fact_sales (grain: one row per order line)
sql/04_checks.sql	Quality checks, one row per check; failed_rows greater than 0 means FAIL
sql/05_metrics.sql	KPI and question views (mart.v_*)

Step 3 - Verify loaded row counts

Row counts after the real-data run:

Table	Rows
clean.orders	159,987
clean.order_items	315,103
clean.payments	164,802
clean.shipping	151,222
clean.returns	15,908
clean.customer_map	45,000
clean.product_map	3,200
audit.dq_log	63
audit.reject	212

Observation: clean.orders holds 159,987 rows while the fact table contains 159,986 distinct orders (the same figure as in the earlier v1 build); the KPI order count of 159,986 is based on orders that have order lines.

Step 4 - Quality checks (04_checks.sql)

All 10 checks returned failed_rows = 0 (10/10 PASS):

#	Check
1	Fact rows equal clean.order_items rows (grain preserved)
2	No duplicate order_item_id in the fact table
3	No NULL surrogate keys in the fact table

#	Check
4	Foreign keys: every fact key exists in its dimension
5	SUM(net_amount) in fact matches the source
6	Shipping fee: sum per order equals the order fee (no double counting)
7	Returned lines are consistent with valid returns
8	Valid returns exist in the fact table
9	No NULL cost amounts
10	Unknown (-1) member exists in every dimension

Note on a script bug found and fixed: run_all.ps1 once reported 'Quality check FAIL' although all 10 checks passed. The cause was PowerShell's -match operator, which ignores letter case, so the word 'fail' inside the column header failed_rows matched. The fix was to use the case-sensitive operator:

```
$res -cmatch "FAIL"
```

Step 5 - Validate the headline KPIs

The KPI view mart.v_kpi_overall was compared with the earlier v1 build and matched exactly. Metric definitions follow the competition brief (NMV includes non-cancelled In Transit orders; Net Revenue = NMV minus valid refunds; Margin = Net Revenue minus COGS on kept lines).

Metric	Value
Gross sales	1,437,241,542.80
NMV	1,199,960,496
Valid refunds	57,668,259
Net Revenue	1,142,292,237
COGS (kept lines)	837,468,444.75
Margin	304,823,792.25 (26.685%)
Orders (total / non-cancelled / cancelled)	159,986 / 145,331 / 14,655
Cancellation rate	9.160%
AOV (NMV / non-cancelled orders)	8,256.74
Units	469,749
Active customers	31,490
Delivered lines / returned lines	283,526 / 14,597
Return rate	5.148%

Step 6 - Business-question views Q1 to Q5

The updated 05_metrics.sql was copied into sql/, applied to the database with ON_ERROR_STOP so any SQL error stops the run, and then queried:

```
Copy-Item "$HOME\Downloads\05_metrics.sql" sql\05_metrics.sql -Force

Get-Content sql\05_metrics.sql -Raw | docker compose exec -T wh psql -U wh -d wh -v ON_ERROR_STOP=1 -q

docker compose exec -T wh psql -U wh -d wh -c "SELECT * FROM mart.v_category_performance ORDER BY revenue_rank"

docker compose exec -T wh psql -U wh -d wh -c "SELECT * FROM mart.v_top5_city_share"
```

Result of mart.v_category_performance (13 rows including the Unknown category, which holds the 932 orphan product lines that were kept, not deleted):

Rank	Category	NMV	Net revenue	Margin %
1	Electronics	427,800,401	413,192,695	8.48
2	Fashion	245,718,757	223,279,883	42.21
3	Home & Kitchen	155,919,619	150,758,459	28.70
4	Mobile & Accessories	108,407,056	104,830,223	41.79
5	Footwear	85,569,937	79,496,913	37.32
6	Beauty & Personal Care	42,020,607	40,555,974	47.31
7	Sports & Outdoors	37,822,539	36,613,820	35.27
8	Grocery	26,165,754	25,358,257	8.91
9	Health & Wellness	22,232,484	21,478,081	41.40
10	Toys & Games	20,424,509	19,755,172	38.78
11	Automotive	15,648,060	15,220,038	29.31
12	Books & Stationery	7,933,355	7,668,743	23.99
13	Unknown	4,297,418	4,083,979	38.40

Cross-checks: (a) the 13 category net_revenue values add up to **1,142,292,237**, exactly the KPI Net Revenue; (b) the top 5 cities have NMV 590,773,148, which is **49.23%** of total NMV (1,199,960,496). Observation for the business questions: Electronics earns the most revenue but has a margin of only about 8.5%, while Fashion has the highest margin among the large categories.

Step 7 - Version control: commit, push, Pull Request, merge

To keep teammate B's modified files out of the commit, the commit uses explicit paths and git status is checked first (only sql/05_metrics.sql must be staged):

```
git add sql/ docs/
git status --short
git commit -m "warehouse: question views Q1-Q5 in 05_metrics.sql" -- sql docs
git push origin warehouse
git status -sb
```

Push succeeded (`ed947bd..c7fefe3 warehouse -> warehouse`) and git status showed no 'ahead'. A Pull Request (#4, base main, compare warehouse) was opened on GitHub, showed no conflicts (1 commit, 1 file, +92 lines) and was merged by the warehouse lead with the pipeline lead's permission; the merge commit on main is 2e012a4. The warehouse branch was kept because further views are still to be written.

5. Design decisions relevant to testing

Decision	Reason
TRUNCATE + CREATE IF NOT EXISTS; fact rebuilt in one transaction	Re-running never creates duplicates and existing views survive (idempotent)
safe_date / safe_num helpers	A bad date or a value like 'Rs. 1,500' becomes NULL instead of crashing the script, so new data with new formats does not break it
ANALYZE after loading	Without it the fact load stalled for more than 4 minutes
Orphan customer or product becomes key -1 (Unknown); lines are never deleted	Keeps totals intact; follows the playbook (empty ship_city_id also becomes -1, no city fallback)
Shipping fee split across lines in proportion to gross	Sum per order equals the order fee, no double counting
Returns aggregated per order line, valid returns only	Preserves the fact table grain
dim_date from 2024-01-01 to 2025-12-31 with an is_order_period flag	About 7% of returns are dated after 30 June 2025; order window ends 30 June 2025

Decision	Reason
Staging tables loaded as TEXT columns	Loading never fails on odd values; conversion to proper types happens in the SQL layer

6. How to re-run the warehouse test

```
cd C:\Users\STAR HIVE\cc26
$env:COMPOSE_FILE="docker-compose.wh-local.yaml"
powershell -ExecutionPolicy Bypass -File sql\run_all.ps1
docker compose exec -T wh psql -U wh -d wh -c "SELECT * FROM mart.v_kpi_overall"
```

With the PostgreSQL container running (Step 1), expected: all 10 quality checks pass and the KPI view returns Net Revenue 1,142,292,237. Stop the container with `docker compose -f docker-compose.wh-local.yaml down` (never with -v).